

Case Study 3: Intelligent Tutoring System

Description: The case base is structured using a list of Python dictionaries to represent past learning experiences. Each case record consists of four primary components to fully capture the context and result of a tutoring intervention:

Student Profile: Contains demographic and identifier data, such as the `student_id` and `grade_level`, to track individual learners.

Problem: Details the specific academic struggle, including the `error_type` (e.g., "sign_error", "formula_misuse"), the math topic, the difficulty level, and a boolean flag indicating if there are repeated errors.

Solution: Outlines the successful intervention previously applied, comprising customized feedback and a specific remedial learning module.

Outcome: Records the success status of the intervention (e.g., "Success" or "Pending Assessment"), ensuring the system improves by retaining accurate new student profiles and outcomes.

The system utilizes a weighted attribute-matching algorithm to calculate a similarity score between the new student's problem and past cases. The scoring logic prioritizes the root cause of the student's struggle over the specific subject matter:

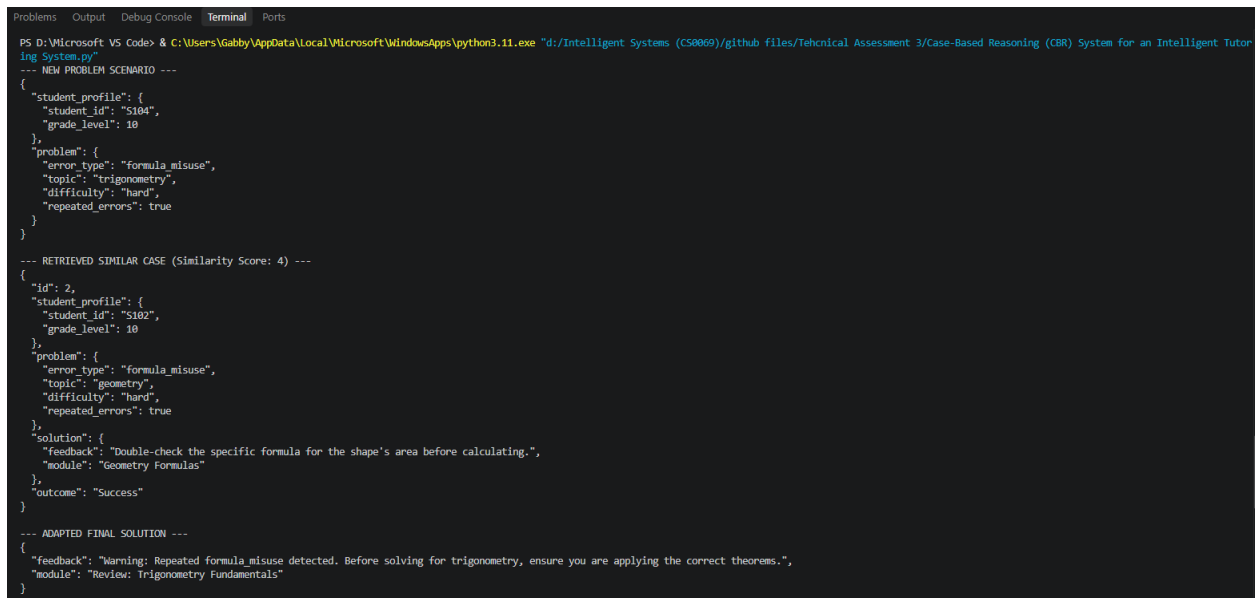
- **Error Type (Weight: +3):** Matching the `error_type` is given the highest priority. Identifying how a student is failing (e.g., misusing formulas) is critical for retrieving an effective pedagogical solution.
- **Topic (Weight: +1):** Matching the specific math subject provides secondary context.
- **Difficulty (Weight: +1):** Matching the problem's difficulty level ensures the retrieved solution is appropriately scaled.

The system iterates through the case base, calculates this score for each past case, and retrieves the case with the highest total score to use as the foundational solution.

Adaptation Logic

Once the most similar past case is retrieved, the adaptation engine modifies the historical solution to fit the exact parameters of the new problem. The logic extracts the core intervention strategy from the past case (e.g., warning the student about a specific error pattern and assigning a fundamental review module) and dynamically injects the new student's current math topic. For example, if a past solution successfully corrected a formula misuse in geometry, the system adapts the feedback and personalized lesson to explicitly reference trigonometry theorems and fundamentals for the new student.

Documentation



```
Problems Output Debug Console Terminal Ports
PS D:\Microsoft VS Code> & C:\Users\Gabby\AppData\Local\Microsoft\WindowsApps\python3.11.exe "d:/Intelligent Systems (CS9009)/github files/Technical Assessment 3/Case-Based Reasoning (CBR) System for an Intelligent Tutoring System.py"
--- NEW PROBLEM SCENARIO ---
{
  "student_profile": {
    "student_id": "S104",
    "grade_level": 10
  },
  "problem": {
    "error_type": "formula_misuse",
    "topic": "trigonometry",
    "difficulty": "hard",
    "repeated_errors": true
  }
}

--- RETRIEVED SIMILAR CASE (Similarity Score: 4) ---
{
  "id": 2,
  "student_profile": {
    "student_id": "S102",
    "grade_level": 10
  },
  "problem": {
    "error_type": "formula_misuse",
    "topic": "geometry",
    "difficulty": "hard",
    "repeated_errors": true
  },
  "solution": {
    "feedback": "Double-check the specific formula for the shape's area before calculating.",
    "module": "Geometry Formulas"
  },
  "outcome": "Success"
}

--- ADAPTED FINAL SOLUTION ---
{
  "feedback": "Warning: Repeated formula_misuse detected. Before solving for trigonometry, ensure you are applying the correct theorems.",
  "module": "Review: Trigonometry Fundamentals"
}
```

- This image displays the initial execution phases: the new problem scenario for Student S104, the retrieval of the most similar past case (Case ID 2 with a score of 4), and the adapted final solution targeting trigonometry.

```
Problems Output Debug Console Terminal Ports

--- NEW CASE SAVED. UPDATED CASE BASE ---
[
  {
    "id": 1,
    "student_profile": {
      "student_id": "S101",
      "grade_level": 9
    },
    "problem": {
      "error_type": "sign_error",
      "topic": "algebra",
      "difficulty": "medium",
      "repeated_errors": true
    },
    "solution": {
      "feedback": "Review the rules for multiplying negative numbers.",
      "module": "Basic Algebra Signs"
    },
    "outcome": "Success"
  },
  {
    "id": 2,
    "student_profile": {
      "student_id": "S102",
      "grade_level": 10
    },
    "problem": {
      "error_type": "formula_misuse",
      "topic": "geometry",
      "difficulty": "hard",
      "repeated_errors": true
    },
    "solution": {
      "feedback": "Double-check the specific formula for the shape's area before calculating.",
      "module": "Geometry Formulas"
    },
    "outcome": "Success"
  },
  {
    "id": 3,
    "student_profile": {
      "student_id": "S103",
      "grade_level": 8
    },
    "problem": {
```

- This shows the beginning of the retention phase, displaying the updated case base array that includes the historical cases (ID 1 and 2) along with their student profiles and successful outcomes.

```

        "error_type": "fraction_addition",
        "topic": "arithmetic",
        "difficulty": "easy",
        "repeated_errors": true
    },
    "solution": {
        "feedback": "Remember to find a common denominator first.",
        "module": "Fraction Fundamentals"
    },
    "outcome": "Success"
},
{
    "id": 4,
    "student_profile": {
        "student_id": "S104",
        "grade_level": 10
    },
    "problem": {
        "error_type": "formula_misuse",
        "topic": "trigonometry",
        "difficulty": "hard",
        "repeated_errors": true
    },
    "solution": {
        "feedback": "Warning: Repeated formula_misuse detected. Before solving for trigonometry, ensure you are applying the correct theorems.",
        "module": "Review: Trigonometry Fundamentals"
    },
    "outcome": "Pending Assessment"
}
]

```

- This displays the end of the updated case base, confirming that the new scenario was successfully appended as Case ID 4 with its adapted solution and an outcome of "Pending Assessment".

SOURCE CODE

Case-Based Reasoning (CBR) System for an Intelligent Tutoring System

```
import json
```

1. Define the Case Base

Storing past cases including student profiles, problems with repeated errors, successful solutions, and outcomes.

```
case_base = [
```

```
{
```

```
    "id": 1,
```

```
    "student_profile": {"student_id": "S101", "grade_level": 9},
```

```
    "problem": {"error_type": "sign_error", "topic": "algebra", "difficulty": "medium",
"repeated_errors": True},
```

```
    "solution": {"feedback": "Review the rules for multiplying negative numbers.", "module":
"Basic Algebra Signs"},
```

```

        "outcome": "Success"
    },
    {
        "id": 2,
        "student_profile": {"student_id": "S102", "grade_level": 10},
        "problem": {"error_type": "formula_misuse", "topic": "geometry", "difficulty": "hard",
"repeated_errors": True},
        "solution": {"feedback": "Double-check the specific formula for the shape's area before
calculating.", "module": "Geometry Formulas"},
        "outcome": "Success"
    },
    {
        "id": 3,
        "student_profile": {"student_id": "S103", "grade_level": 8},
        "problem": {"error_type": "fraction_addition", "topic": "arithmetic", "difficulty": "easy",
"repeated_errors": True},
        "solution": {"feedback": "Remember to find a common denominator first.", "module":
"Fraction Fundamentals"},
        "outcome": "Success"
    }
]

```

The new student profile and problem scenario

```
new_student_profile = {"student_id": "S104", "grade_level": 10}
```

```
new_problem = {"error_type": "formula_misuse", "topic": "trigonometry", "difficulty": "hard",
"repeated_errors": True}
```

```
print("--- NEW PROBLEM SCENARIO ---")

print(json.dumps({"student_profile": new_student_profile, "problem": new_problem},
indent=2))
```

2. Similarity Assessment

```
def calculate_similarity(new_prob, past_prob):

    score = 0

    if new_prob["error_type"] == past_prob["error_type"]: score += 3 # High weight for error type
    if new_prob["topic"] == past_prob["topic"]: score += 1
    if new_prob["difficulty"] == past_prob["difficulty"]: score += 1

    return score
```

```
best_match = None
```

```
highest_score = -1
```

```
for case in case_base:

    score = calculate_similarity(new_problem, case["problem"])

    if score > highest_score:

        highest_score = score

        best_match = case
```

```
print(f"\n--- RETRIEVED SIMILAR CASE (Similarity Score: {highest_score}) ---")

print(json.dumps(best_match, indent=2))
```

3. Adapt the Solution

```
# Modifying the retrieved solution to fit the new problem's specific topic context.
```

```
adapted_solution = best_match["solution"].copy()

adapted_solution["feedback"] = f"Warning: Repeated {new_problem['error_type']} detected.
Before solving for {new_problem['topic']}, ensure you are applying the correct theorems."

adapted_solution["module"] = f"Review: {new_problem['topic'].capitalize()} Fundamentals"

print("\n--- ADAPTED FINAL SOLUTION ---")

print(json.dumps(adapted_solution, indent=2))
```

4. Demonstrate Learning (Retention)

Saving the newly solved case into the system's case base, retaining the new student profile and pending outcome.

```
new_case = {
    "id": len(case_base) + 1,
    "student_profile": new_student_profile,
    "problem": new_problem,
    "solution": adapted_solution,
    "outcome": "Pending Assessment"
}

case_base.append(new_case)

print("\n--- NEW CASE SAVED. UPDATED CASE BASE ---")

print(json.dumps(case_base, indent=2))
```